Medical Image Classification Midterm Report

Ikechukwu Mgbemele | Senior Seminar I | Fisk University | Weeks 1 to 7 | October 5, 2026

Project overview

My project looks at whether uncertainty estimation methods can help identify wrong predictions in medical image classification. A model may perform well on clear images but struggle when images have noise, blur, or lower contrast. I want to find out whether its confidence gives a useful warning when this happens.

So far, I have trained the classifier, tested it on altered images, and implemented all three uncertainty methods. The main finding is that accuracy drops much faster than confidence. The methods reduce this gap, but the model still makes many wrong predictions with high confidence.

Progress so far

Milestone	Status
1 Set up the project and train the baseline model	Complete
2 Add noise, blur, and contrast changes to test images	Complete
3 Implement the three uncertainty methods	Complete
4 Measure error detection and calibration	Not started
5 Compare results and write the final analysis	Not started

Training the baseline model

I used PathMNIST, a public dataset with nine tissue classes from colorectal cancer slides. It contains 89,996 training images, 10,004 validation images, and 7,180 test images. I chose the 64 × 64 version because it gives me more image detail to work with when adding blur.

The test images come from a different hospital than the training and validation images. This lets me check how the model handles a change in the data even before I alter the images.

I built a convolutional neural network using Python 3.11, TensorFlow 2.18, and Keras 3. It has four convolution blocks and about 1.24 million parameters. Training took 40 epochs, or about 165 minutes. Epoch 34 had the lowest validation loss. I included dropout during training and kept the raw model scores so I could use them for the uncertainty methods later.

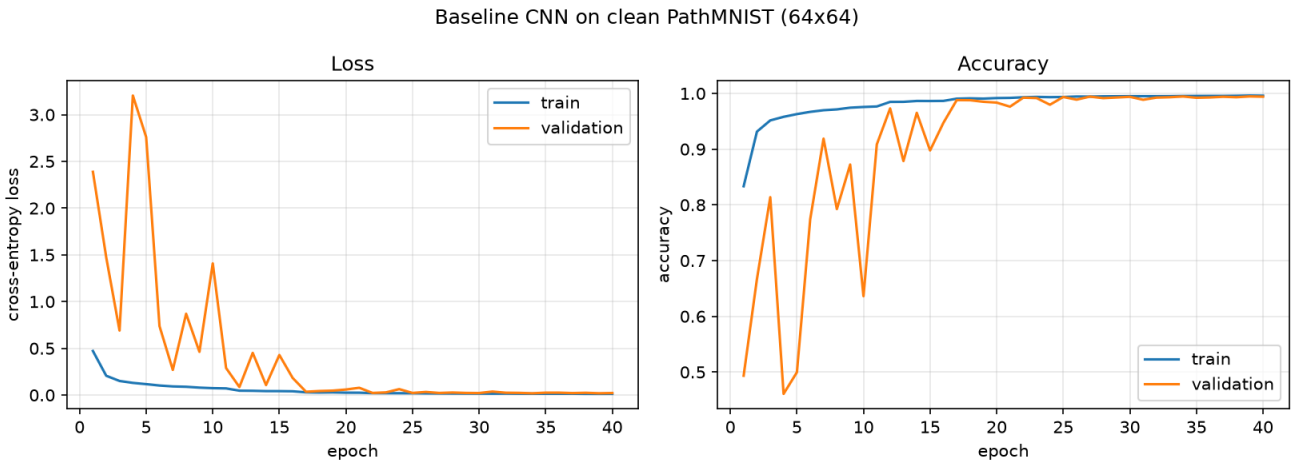


Figure 1. Training accuracy and loss over 40 epochs. Validation results varied early in training and became more stable after the learning rate was reduced.

Baseline results

Metric	Validation	Clean test
Accuracy	99.45%	93.48%
Balanced accuracy	99.45%	91.15%
Macro F1	0.9944	0.9052
Mean confidence	99.64%	96.97%

On validation images, average confidence was close to accuracy. On the test images, confidence was about 3.5 percentage points higher than accuracy. Of the 468 wrong test predictions, 96 had confidence above 99%. This shows why confidence alone may not be enough to judge whether a prediction is reliable.

The model struggled most with cancer-associated stroma. It correctly identified about 58% of these images and often confused them with debris or smooth muscle.

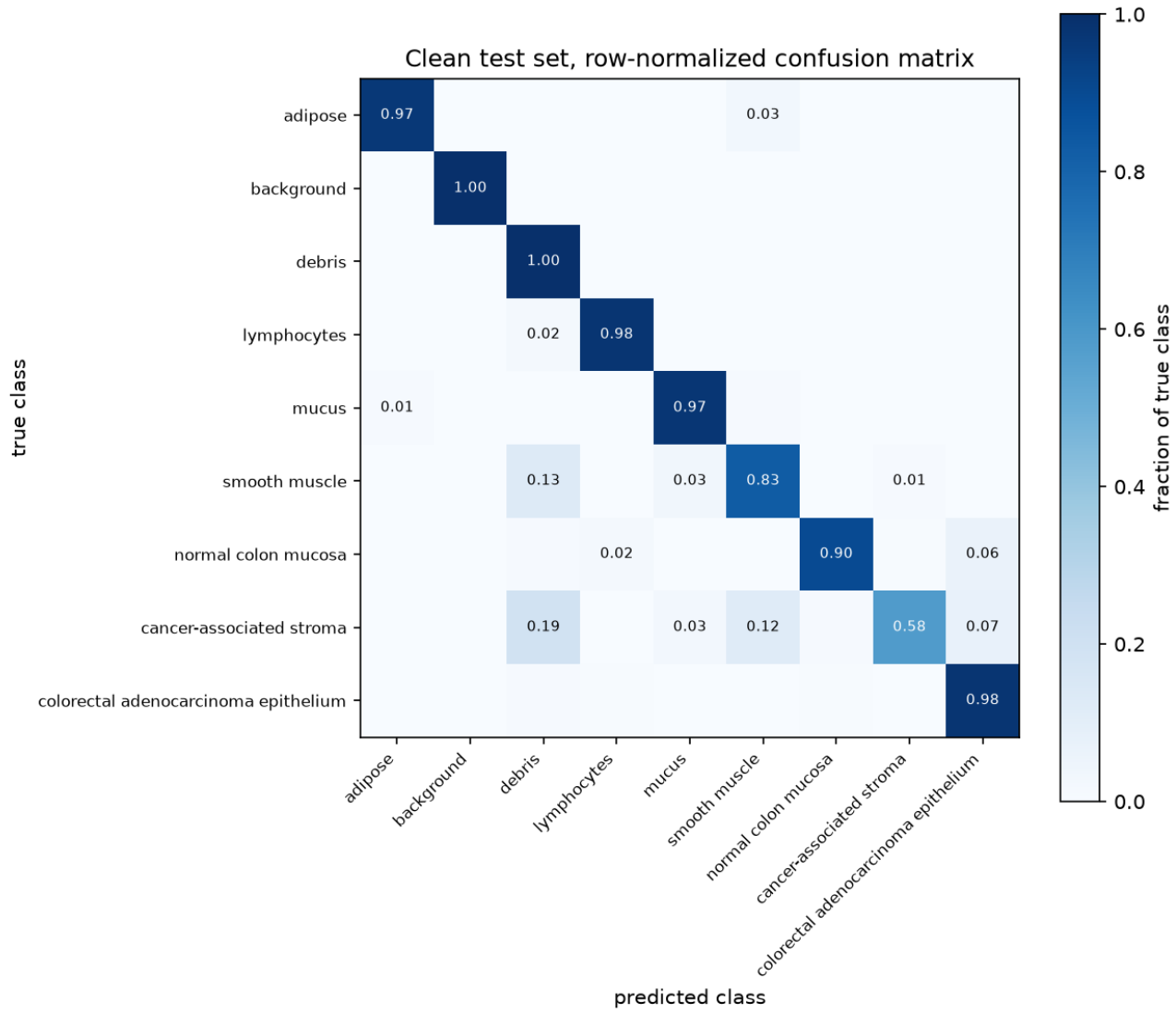


Figure 2. Confusion matrix for the clean test set. The model correctly identified about 58% of cancer-associated stroma images and often confused them with debris or smooth muscle.

Testing altered images

I added Gaussian noise, Gaussian blur, and reduced contrast to the test images only. I used five severity levels for each change. I kept the training data unchanged and used fixed random seeds so each uncertainty method would receive the same altered images.

Choosing the severity levels took more work than I expected. My first noise settings were too strong and pushed several levels to around chance accuracy. I wrote a script to test a wider range of settings on 2,500 images, then chose levels that gave a gradual drop in performance. I aimed for a lowest accuracy of about 40% on this sample. Results on the full test set were sometimes lower.

Image change	Levels 1 to 5
Noise standard deviation	0.02, 0.025, 0.03, 0.035, 0.04
Blur standard deviation in pixels	0.6, 0.8, 0.9, 1.2, 1.6

Image change	Levels 1 to 5
Contrast factor	0.85, 0.8, 0.7, 0.65, 0.6

The table below shows the results at the strongest selected level. The clean test accuracy was 93.48%, with average confidence of 96.97%.

Image change	Accuracy	Mean confidence
Noise	32.4%	87.8%
Blur	37.4%	85.7%
Reduced contrast	32.8%	81.9%

Confidence stayed high even when accuracy dropped sharply. It sometimes increased as accuracy fell. Under the strongest noise, 1,644 errors had confidence above 99%. Noise and blur increased the share of these highly confident errors, while reduced contrast lowered that share. The model responded differently to each type of image change.

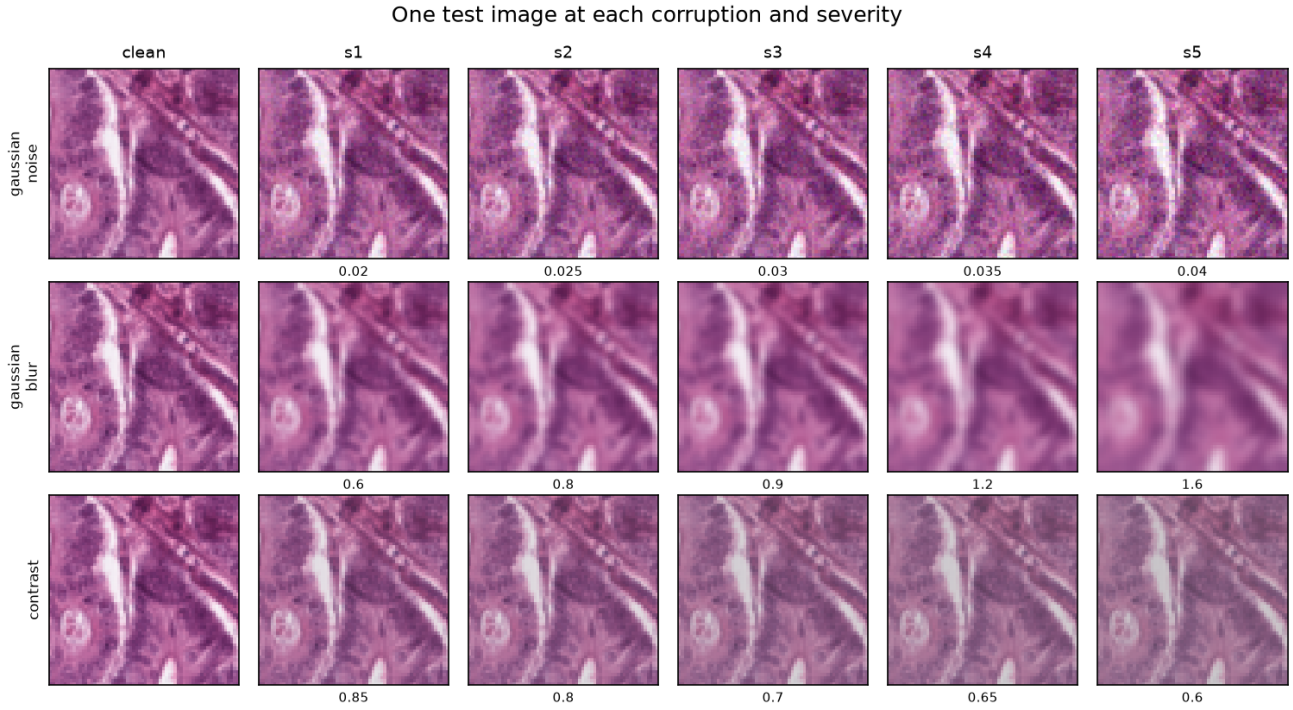


Figure 3. Examples of noise, blur, and reduced contrast at each severity level. These show how the test images change as severity increases.

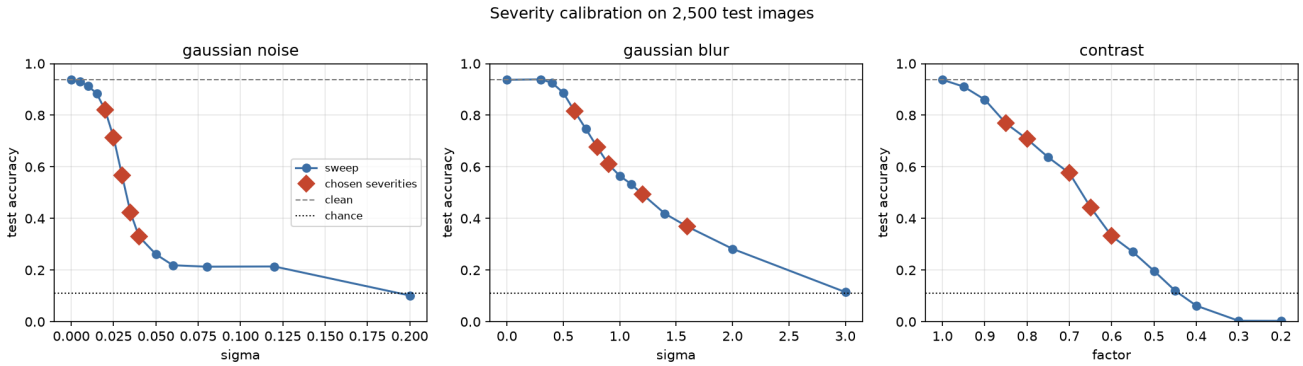


Figure 4. Severity settings tested on 2,500 images. Red diamonds mark the selected levels, and the dotted line shows chance accuracy.

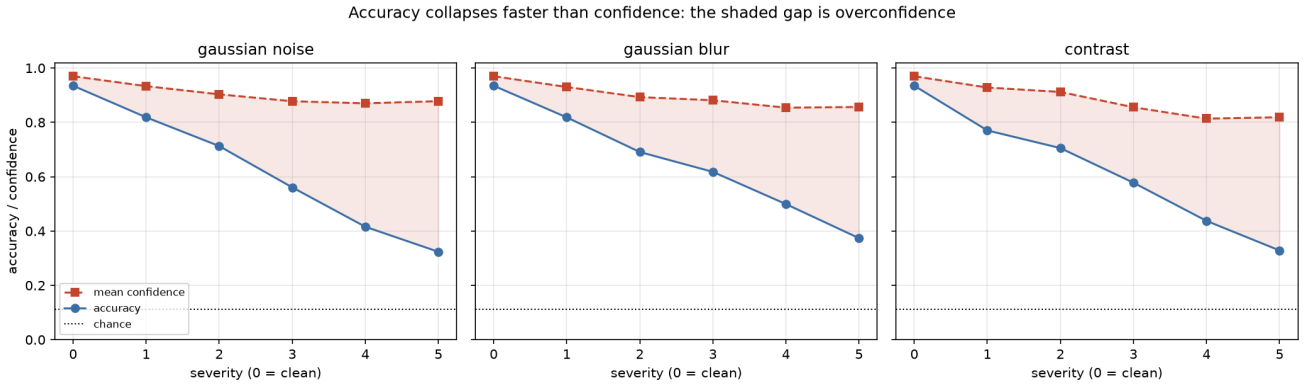


Figure 5. Accuracy and average confidence at each severity level. The shaded area shows the gap between them, which grows as the images become more altered.

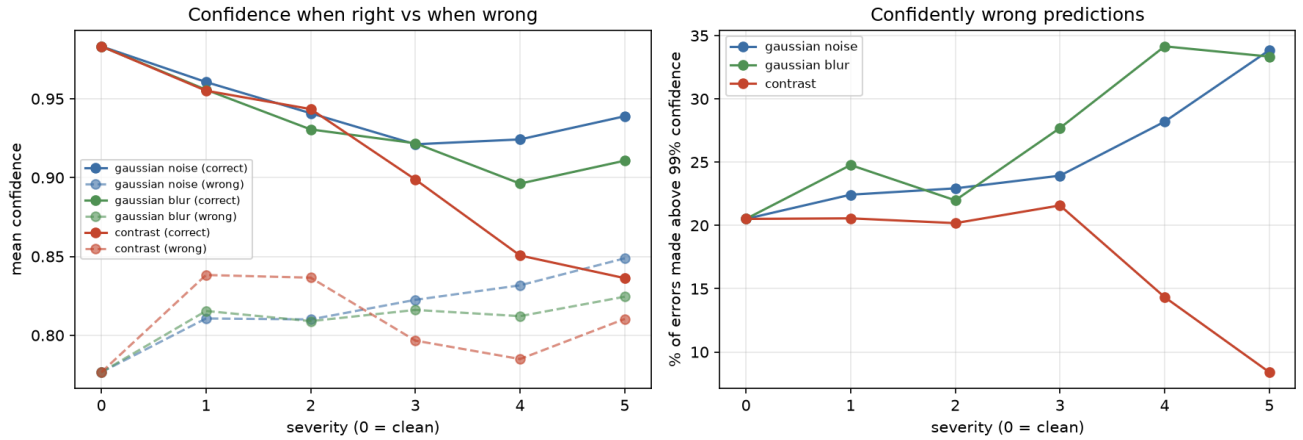


Figure 6. Left: average confidence for correct and wrong predictions. Right: the share of errors with confidence above 99%. Noise and blur increase this share, while reduced contrast lowers it.

Comparing uncertainty methods

I implemented softmax confidence, temperature scaling, and Monte Carlo Dropout. Softmax uses the model's usual confidence score. Temperature scaling adjusts the raw scores to change confidence without changing the predicted class. Monte Carlo Dropout runs the model several times with dropout active and averages the results.

For temperature scaling, I fitted a temperature of 1.2759 using half of the validation set and checked it on the other half. I did not use the test set to fit it. For Monte Carlo Dropout, I used 30 passes per image. I also measured mutual information, which captures how much those passes disagree.

Both temperature scaling and Monte Carlo Dropout reduced the average gap between confidence and accuracy compared with softmax. The table shows this gap at severity 5, in percentage points. A smaller gap means average confidence is closer to accuracy, but it does not by itself prove that a method can identify individual errors.

Image change	Softmax	Temperature	MC Dropout
Noise	55.4	52.1	43.9
Blur	48.3	44.1	40.3
Reduced contrast	49.0	43.5	41.8

Monte Carlo Dropout gave the smallest gap under noise at every severity. Under blur and reduced contrast, the better method depended on severity. None of the methods fully solved the overconfidence problem.

Temperature scaling left the predicted classes unchanged. Monte Carlo Dropout changed some predictions and raised accuracy under the strongest noise from 32.4% to 37.0%. However, it also lowered accuracy in some blur and contrast settings.

Mutual information increased with severity for all three image changes. This makes it worth testing as a possible warning signal, though I still need to measure how well it identifies wrong predictions.

I also checked whether temperature scaling changed the order of confidence scores. In these preliminary checks, the scores stayed almost in the same order, and error-detection AUROC changed by less than 0.005. The full comparison is still part of Milestone 4. Adjusting confidence and detecting errors are different tasks.

As an extra check, I fitted temperatures directly on individual test conditions. This uses labels that would not normally be available, so it is only a reference. Even this approach left a confidence gap under strong noise, suggesting that one temperature may not be enough for badly altered images.

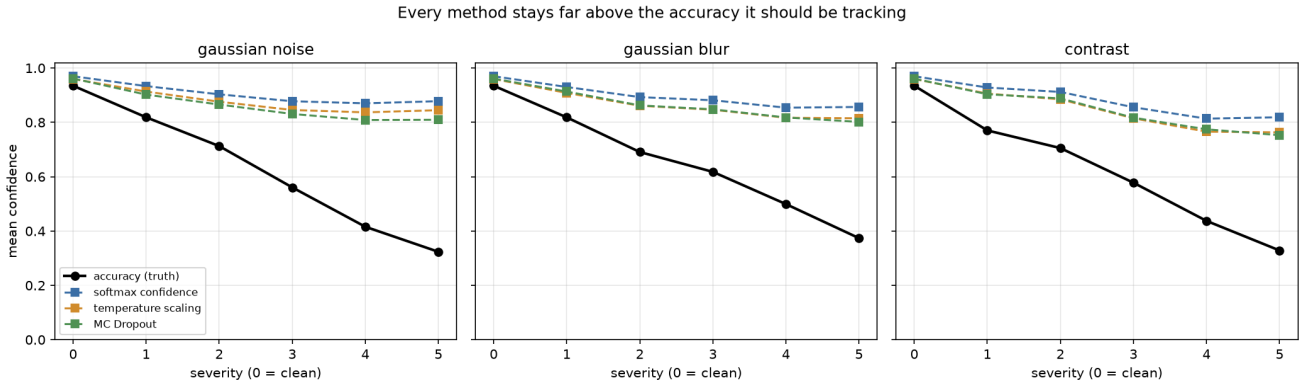


Figure 7. Average confidence from each uncertainty method compared with accuracy, shown in black. Confidence remains above accuracy under the image changes.

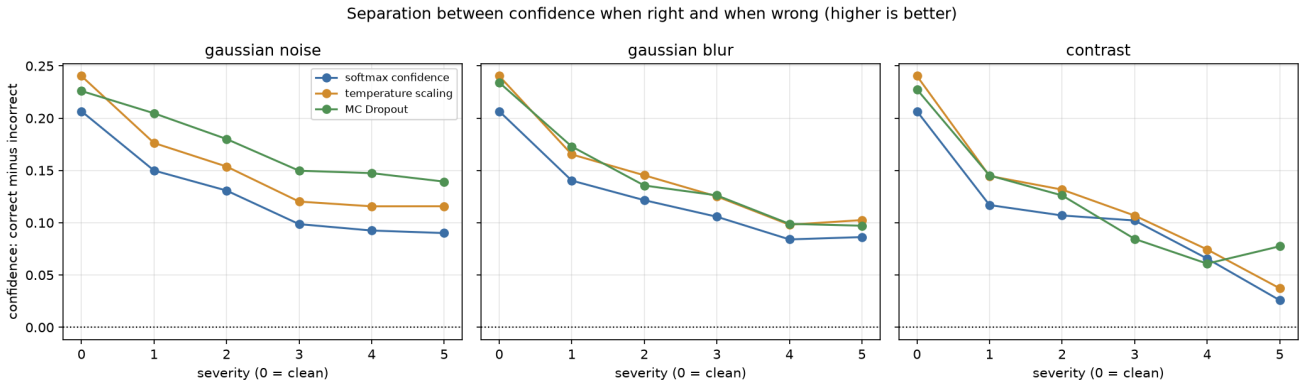


Figure 8. The difference in average confidence between correct and wrong predictions. Monte Carlo Dropout keeps the largest difference under noise, while all three methods struggle under strong contrast reduction.

Saved work

The project is tracked in Git. I have 11 Python modules, eight figures, and six metrics files covering training, baseline evaluation, image changes, and the uncertainty methods. I also saved the predictions and probability scores so I can calculate the remaining metrics without running the model again.

Challenges and solutions

Environment setup was an early challenge. TensorFlow did not work with the Python versions I first tried, and the GPU plugin was incompatible with the newer TensorFlow version. I switched to Python 3.11 and TensorFlow 2.18 with tensorflow-metal 1.2.0. I checked CPU and GPU outputs before continuing. Predicted classes matched, and numerical differences were very small.

During training, validation accuracy stayed near chance even though training accuracy was improving. After checking the data and model behavior, I traced this to Batch Normalization's stored statistics. Reducing its momentum from 0.99 to 0.9 fixed the issue. In a two-epoch check, validation accuracy rose from 10.9% to 64.0%.

The severity settings also needed adjustment. Some were too strong, and one selection attempt chose unchanged images as the first contrast level. I updated the calibration script to choose progressively stronger changes without reusing the clean setting.

For Monte Carlo Dropout, calling the model in training mode would also change how Batch Normalization works. I replaced only the dropout layers so dropout could stay active while Batch Normalization remained in inference mode. A check with dropout set to zero reproduced the original model's output exactly.

Repeated Monte Carlo Dropout runs still varied on the GPU, even with a fixed seed. Across five seeds, accuracy ranged from 94.15% to 94.50%. I will account for this variation when interpreting small differences between methods.

Progress compared with the proposal

I have completed the first three milestones and am on schedule. I followed the proposed approach of training a medical image classifier, altering only test images, and comparing the three uncertainty methods. The formal error-detection and calibration measurements are still incomplete.

I trained a smaller model from scratch, which the proposal allowed. I also added severity calibration and the extra temperature checks because they helped me interpret the results. I left contrast and brightness changes out of training augmentation so they would not overlap with the changes being studied.

Next steps

Next, I will calculate error-detection AUROC to measure how well each method ranks wrong predictions above correct ones. I will also calculate expected calibration error and create reliability diagrams to check how closely confidence matches accuracy.

I will test whether removing the least confident predictions improves accuracy and compare mutual information with the usual confidence score. Then I will bring the results together and write the final analysis, including the cost of running Monte Carlo Dropout 30 times per image.

The study uses one model and one dataset, and the image changes are artificial. The selected severity levels also depend on this model. I will keep these limits clear when explaining what the results show.